Experiment 11

Title: Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

Aim: To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

Theory:

Hyperledger Fabric is an open-source, permissioned blockchain platform developed for enterprise-level applications. Unlike public blockchains, where anyone can participate, Hyperledger Fabric restricts access to authorized users, providing enhanced security, privacy, and control over data. This makes it highly suitable for business environments where confidentiality and trust are critical.

Hyperledger Fabric follows a modular and flexible architecture. Its key components include **peers**, which maintain the distributed ledger and execute transactions; the **ordering service**, which ensures that all transactions are arranged in a consistent order; and **channels**, which allow a subset of network participants to create private communication layers for confidential transactions. This channel-based approach ensures that sensitive business data is only shared among relevant parties.

A core concept in Hyperledger Fabric is **chaincode**, which represents the business logic of the application. Chaincode is similar to smart contracts in other blockchain platforms and is typically written in programming languages like Go, Java, or JavaScript. It defines how assets are created, updated, transferred, and queried on the ledger. When a transaction is initiated, the chaincode is executed by endorsing peers, and the result is recorded on the ledger after validation.

The ledger in Hyperledger Fabric consists of two parts: the **blockchain**, which is an immutable record of all transactions, and the **world state**, which stores the latest state of assets for quick access. Assets can represent any tangible or intangible entity such as products, documents, or financial records, and they are managed efficiently through chaincode operations.

This framework ensures important features such as transparency, immutability, and security, while also providing high performance and scalability. As a result, Hyperledger Fabric is widely used in real-world applications like supply chain management, banking, healthcare, and digital identity systems, where secure and efficient asset management is essential.

Implementation:

Step 1: Update the System
sudo apt update
sudo apt upgrade -y

```
g2022_mahvish_siddiqui@cloudshell:~$ sudo apt update
sudo apt upgrade -y
Get:1 https://cli.github.com/packages stable InRelease [3,917 B]
Get:2 https://apt.postgresql.org/pub/repos/apt noble-pgdg InRelease [180 kB]
Get:3 https://apt.postgresql.org/pub/repos/apt noble-pgdg/main amd64 Packages [675 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:5 https://ppa.launchpadcontent.net/dotnet/backports/ubuntu noble InRelease
Get:6 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1,989 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1,508 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [3,505 kB]
Get:9 https://download.docker.com/linux/ubuntu noble InRelease [48.5 kB]
Get:10 https://packages.cloud.google.com/apt gcsfuse-noble InRelease [1,227 B]
Get:11 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [59.7 kB]
Get:12 https://packages.cloud.google.com/apt cloud-sdk InRelease [1,620 B]
Get:13 https://packages.cloud.google.com/apt gcsfuse-noble/main amd64 Packages [66.1 kB]
Get:14 https://packages.cloud.google.com/apt cloud-sdk/main all Packages [1,982 kB]
Get:15 https://packages.cloud.google.com/apt cloud-sdk/main amd64 Packages [4,640 kB]
Get:16 https://repo.mongodb.org/apt/ubuntu noble/mongodb-org/8.0 InRelease [3,005 B]
Hit:17 http://archive.ubuntu.com/ubuntu noble InRelease
Get:18 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:19 https://repo.mongodb.org/apt/ubuntu noble/mongodb-org/8.0/multiverse amd64 Packages [68.7 kB]
Get:20 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:21 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Packages [3,692 kB]
```

Step 2: Install Required Dependencies

sudo apt install jq curl git nodejs npm -y

node -v

npm -v

git --version

docker --version

docker ps

```
g2022_mahvish_siddiqui@cloudshell:~$ sudo apt install -y jq curl
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
jq is already the newest version (1.7.1-3ubuntu0.24.04.1).
curl is already the newest version (8.5.0-2ubuntu10.8).
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.
g2022_mahvish_siddiqui@cloudshell:~$ sudo apt install -y jq git
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
jq is already the newest version (1.7.1-3ubuntu0.24.04.1).
git is already the newest version (1:2.43.0-1ubuntu7.3).
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.
g2022_mahvish_siddiqui@cloudshell:~$
```

```
g2022_mahvish_siddiqui@cloudshell:~$ node -v
v24.14.1
g2022_mahvish_siddiqui@cloudshell:~$ npm -v
11.12.1
g2022_mahvish_siddiqui@cloudshell:~$ git --version
git version 2.43.0
g2022_mahvish_siddiqui@cloudshell:~$ docker --v
unknown flag: --v

Usage:  docker [OPTIONS] COMMAND [ARG...]

Run 'docker --help' for more information
g2022_mahvish_siddiqui@cloudshell:~$ docker --version
Docker version 29.4.0, build 9d7ad9f
g2022_mahvish_siddiqui@cloudshell:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
g2022_mahvish_siddiqui@cloudshell:~$
```

Step 3: Clone Fabric Samples

git clone https://github.com/hyperledger/fabric-samples

cd fabric-samples

```
g2022_mahvish_siddiqui@cloudshell:~$ cd $HOME
git clone https://github.com/hyperledger/fabric-samples
cd fabric-samples
Cloning into 'fabric-samples'...
remote: Enumerating objects: 15618, done.
remote: Counting objects: 100% (159/159), done.
remote: Compressing objects: 100% (101/101), done.
remote: Total 15618 (delta 95), reused 58 (delta 58), pack-reused 15459 (from 4)
Receiving objects: 100% (15618/15618), 24.18 MiB | 16.03 MiB/s, done.
Resolving deltas: 100% (8760/8760), done.
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples$
```

Step 4: Install Fabric Binaries and Images

curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh

chmod +x install-fabric.sh

./install-fabric.sh docker samples binary

./install-fabric.sh docker

```
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples$ curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh docker

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.15 does not exist, defaulting to main. fabric-samples main branch is intended to work with recent versions of fabric.

Pull Hyperledger Fabric binaries

===> Downloading version 2.5.15 platform specific fabric binaries
===> Downloading:  https://github.com/hyperledger/fabric/releases/download/v2.5.15/hyperledger-fabric-linux-amd64-2.5.15.tar.gz
===> Will unpack to: /home/g2022_mahvish_siddiqui/fabric-samples
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
100  120M  100  120M    0     0  28.7M      0  0:00:04  0:00:04 --:--:-- 31.0M
==> Done.
===> Downloading version 1.5.17 platform specific fabric-ca-client binary
===> Downloading:  https://github.com/hyperledger/fabric-ca/releases/download/v1.5.17/hyperledger-fabric-ca-linux-amd64-1.5.17.tar.gz
===> Will unpack to: /home/g2022_mahvish_siddiqui/fabric-samples
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0
100 31.6M  100 31.6M    0     0  22.3M      0  0:00:01  0:00:01 --:--:-- 41.8M
==> Done.
```

```
Status: Image is up to date for ghcr.io/hyperledger/fabric-baseos:2.5.15
ghcr.io/hyperledger/fabric-baseos:2.5.15
===> Pulling fabric ca Image
====>  ghcr.io/hyperledger/fabric-ca:1.5.17
1.5.17: Pulling from hyperledger/fabric-ca
Digest: sha256:453a0a727e82c4960b797e379adc231e853ee23ae7526db53afef77b5074117e
Status: Image is up to date for ghcr.io/hyperledger/fabric-ca:1.5.17
ghcr.io/hyperledger/fabric-ca:1.5.17
===> List out hyperledger images
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15    654bd4554bf9        250MB         80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17        453a0a727e82        381MB          123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15     86dded7eda92          1GB          255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15   9972c209be47        178MB         49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15      3e25adc31a75        230MB         66.2MB
hyperledger/fabric-baseos:2.5               654bd4554bf9        250MB         80.4MB
hyperledger/fabric-baseos:2.5.15            654bd4554bf9        250MB         80.4MB
hyperledger/fabric-baseos:latest            654bd4554bf9        250MB         80.4MB
hyperledger/fabric-ca:1.5                   453a0a727e82        381MB          123MB
hyperledger/fabric-ca:1.5.17                453a0a727e82        381MB          123MB
hyperledger/fabric-ca:latest                453a0a727e82        381MB          123MB
hyperledger/fabric-ccenv:2.5                86dded7eda92          1GB          255MB
hyperledger/fabric-ccenv:2.5.15             86dded7eda92          1GB          255MB
hyperledger/fabric-ccenv:latest             86dded7eda92          1GB          255MB
hyperledger/fabric-orderer:2.5              9972c209be47        178MB         49.5MB
hyperledger/fabric-orderer:2.5.15           9972c209be47        178MB         49.5MB
hyperledger/fabric-orderer:latest           9972c209be47        178MB         49.5MB
hyperledger/fabric-peer:2.5                 3e25adc31a75        230MB         66.2MB
hyperledger/fabric-peer:2.5.15              3e25adc31a75        230MB         66.2MB
hyperledger/fabric-peer:latest              3e25adc31a75        230MB         66.2MB
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples$
```

Step 5: Verify Docker Images

docker images | grep fabric

```
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples$ docker images | grep fabric
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15    654bd4554bf9        250MB         80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17        453a0a727e82        381MB          123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15     86dded7eda92          1GB          255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15   9972c209be47        178MB         49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15      3e25adc31a75        230MB         66.2MB
hyperledger/fabric-baseos:2.5               654bd4554bf9        250MB         80.4MB
hyperledger/fabric-baseos:2.5.15            654bd4554bf9        250MB         80.4MB
hyperledger/fabric-baseos:latest            654bd4554bf9        250MB         80.4MB
hyperledger/fabric-ca:1.5                   453a0a727e82        381MB          123MB
hyperledger/fabric-ca:1.5.17                453a0a727e82        381MB          123MB
hyperledger/fabric-ca:latest                453a0a727e82        381MB          123MB
hyperledger/fabric-ccenv:2.5                86dded7eda92          1GB          255MB
hyperledger/fabric-ccenv:2.5.15             86dded7eda92          1GB          255MB
hyperledger/fabric-ccenv:latest             86dded7eda92          1GB          255MB
hyperledger/fabric-orderer:2.5              9972c209be47        178MB         49.5MB
hyperledger/fabric-orderer:2.5.15           9972c209be47        178MB         49.5MB
hyperledger/fabric-orderer:latest           9972c209be47        178MB         49.5MB
hyperledger/fabric-peer:2.5                 3e25adc31a75        230MB         66.2MB
hyperledger/fabric-peer:2.5.15              3e25adc31a75        230MB         66.2MB
hyperledger/fabric-peer:latest              3e25adc31a75        230MB         66.2MB
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples$
```

Step 6: Set Environment Variables

export PATH=$PATH:$HOME/fabric-samples/bin

export FABRIC_CFG_PATH=$HOME/fabric-samples/config

peer version

```
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples$ export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
peer version
peer:
 Version: v2.5.15
 Commit SHA: 83c7930
 Go version: go1.26.0
 OS/Arch: linux/amd64
 Chaincode:
  Base Docker Label: org.hyperledger.fabric
  Docker Namespace: hyperledger
```

Step 7: Start Network and Create Channel

cd ~/fabric-samples/test-network

./network.sh up createChannel

```
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples$ cd ~/fabric-samples/test-network
./network.sh up createChannel
Using docker and docker-compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using database 'leveld
Bringing up network
LOCAL_VERSION=v2.5.15
DOCKER_IMAGE_VERSION=v2.5.15
/home/g2022_mahvish_siddiqui/fabric-samples/test-network/../bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org1.yaml --output=organizations
org1.example.com
+ res=0
Creating Org2 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org2.yaml --output=organizations
org2.example.com
+ res=0
Creating Orderer Org Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-orderer.yaml --output=organizations
+ res=0
Generating CCP files for Org1 and Org2
ERROR: The Compose file './compose/compose-test-net.yaml' is invalid because:
Unsupported config option for services: 'peer0.org1.example.com'
Unsupported config option for volumes: 'peer0.org1.example.com'
Unsupported config option for networks: 'test'
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
Using docker and docker-compose
```

```
++ echo '{"payload":{"header":{"channel_header":{"channel_id":"mychannel", "type":2}},"data":{"config_update":{' '"channel_id":' '"mychannel",' '"isolated_data":' '{},' '"read
_set":' '{' '"groups":' '{' '"Application":' '{' '"groups":' '{' '"Org2MSP":' '{' '"groups":' '{},' '"mod_policy":' '"",' '"policies":' '{' '"Admins":' '{' '"mod_policy":' '"
",' '"policy":' null, '"version":' '"0"' '},' '"Endorsement":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0"' '},' '"Readers":' '{' '"mod_policy":' '"",' '"poli
cy":' null, '"version":' '"0"' '},' '"Writers":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0"' '}' '},' '"values":' '{' '"MSP":' '{' '"mod_policy":' '"",' '"v
alue":' null, '"version":' '"0"' '}' '},' '"version":' '"0"' '}' '},' '"mod_policy":' '"",' '"policies":' '{},' '"values":' '{},' '"version":' '"0"' '}' '},' '"mod_policy":' '
"",' '"policies":' '{},' '"values":' '{},' '"version":' '"0"' '},' '"write_set":' '{' '"groups":' '{' '"Application":' '{' '"groups":' '{' '"Org2MSP":' '{' '"groups":' '{},' '
"mod_policy":' '"Admins",' '"policies":' '{' '"Admins":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0"' '},' '"Endorsement":' '{' '"mod_policy":' '"",' '"polic
y":' null, '"version":' '"0"' '},' '"Readers":' '{' '"mod_policy":' '"",' '"policy":' null, '"version":' '"0"' '},' '"Writers":' '{' '"mod_policy":' '"",' '"policy":' null, '"
version":' '"0"' '}' '},' '"values":' '{' '"AnchorPeers":' '{' '"mod_policy":' '"Admins",' '"value":' '{' '"anchor_peers":' '[' '{' '"host":' '"peer0.org2.example.com",' '"por
t":' 9051 '}' ']' '},' '"version":' '"0"' '},' '"MSP":' '{' '"mod_policy":' '"",' '"value":' null, '"version":' '"0"' '}' '},' '"version":' '"1"' '}' '},' '"mod_policy":' '"",
' '"policies":' '{},' '"values":' '{},' '"version":' '"0"' '}' '},' '"mod_policy":' '"",' '"policies":' '{},' '"values":' '{},' '"version":' '"0"' '}' '}}}}'
++ configtxlator proto_encode --input /home/g2022_mahvish_siddiqui/fabric-samples/test-network/channel-artifacts/config_update_in_envelope.json --type common.Envelope --output
 /home/g2022_mahvish_siddiqui/fabric-samples/test-network/channel-artifacts/Org2MSPanchors.tx
2026-04-09 20:26:51.128 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
2026-04-09 20:26:51.155 UTC 0002 INFO [channelCmd] update -> Successfully submitted channel update
Anchor peer set for org 'Org2MSP' on channel 'mychannel'
Channel 'mychannel' joined
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$
```

Step 8: Deploy Chaincode

```
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$ ./network.sh deployCC \
 -ccn basic \
 -ccp ../asset-transfer-basic/chaincode-javascript \
 -ccl javascript
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/chaincode-javascript --lang node --label basic_1.0
+ res=0
Chaincode is packaged
```

Step 9: Set Peer Environment

export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true

export CORE_PEER_LOCALMSPID="Org1MSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051

```
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$ export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"

export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt

export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp

export CORE_PEER_ADDRESS=localhost:7051
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$
```

Step 10: Initialize Ledger

peer chaincode invoke ... InitLedger

```
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$ peer chaincode invoke \
 -o localhost:7050 \
 --ordererTLSHostnameOverride orderer.example.com \
 --tls \
 --cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.exam
 -C mychannel \
 -n basic \
 --peerAddresses localhost:7051 \
 --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \
 --peerAddresses localhost:9051 \
 --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \
 -c '{"function":"InitLedger","Args":[]}'
2026-04-09 20:34:03.243 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$
```

Step 11: Query All Assets

```
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n
 basic -c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"},{"Ap
praisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"},{"AppraisedV
alue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"},{"AppraisedVal
ue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"},{"AppraisedValue":7
00,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"},{"AppraisedValue":800
,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$
```

Step 12: Read Specific Asset

```
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n
 basic -c '{"Args":["ReadAsset","asset5"]}'
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$
```

Step 13: Create New Asset

peer chaincode invoke ... CreateAsset (Owner: Manav)

```
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$
```

Step 14: Transfer Asset

peer chaincode invoke ... TransferAsset

```
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile $HOME/
fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem -C mychannel -n basic --peerA
ddresses localhost:7051 --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt --peerAdd
resses localhost:9051 --tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt -c '{"funct
ion":"InitLedger","Args":[]}'
2026-04-09 20:41:46.211 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
```

Step 15: Stop Network

./network.sh down

```
g2022_mahvish_siddiqui@cloudshell:~/fabric-samples/test-network$ cd $HOME/fabric-samples/test-network
./network.sh down
Using docker and docker-compose
Stopping network
[+] down 7/7
 ✓ Container peer0.org1.example.com      Removed                                                                    0.7s
 ✓ Container peer0.org2.example.com      Removed                                                                    0.8s
 ✓ Container orderer.example.com         Removed                                                                    0.6s
 ✓ Volume compose_peer0.org1.example.com Removed                                                                    0.0s
 ✓ Volume compose_peer0.org2.example.com Removed                                                                    0.0s
 ✓ Volume compose_orderer.example.com    Removed                                                                    0.0s
 ✓ Network fabric_test                   Removed                                                                    0.1s
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59b629-c68f8ecc2a0ff0cc1497615f5e33a8bfa908e0fd3555f864cfa1398b52d341c
b:latest
Deleted: sha256:12e3faf2b9ca30a7182645a005f0d09fe0fae610612b1f2bdfbe2799a1024009
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59b629-d3cf9b1f4b9747cc8433ac0e74bfaeade1cf0b97f5c1163835f604aa885d60d
c:latest
Deleted: sha256:6fe9622fd07b0e414e4de896b19dae20f5b8312d5703581ca75bcc31a4196098
Unable to find image 'busybox:latest' locally
latest: Pulling from library/busybox
481282afbc43: Pull complete
aecba0016ef2: Download complete
Digest: sha256:1487d0af5f52b4ba31c7e465126ee2123fe3f2305d638e7827681e7cf6c83d5e
```

Conclusion:

The experiment successfully demonstrated the implementation of a Hyperledger Fabric network along with deploying and interacting with chaincode for asset management. Through this process, we were able to create, query, and transfer assets efficiently, showcasing the core functionalities of a permissioned blockchain. It provided practical understanding of how transactions are executed, validated, and recorded securely within the network. Overall, the experiment highlighted the reliability, transparency, and control offered by Hyperledger Fabric for managing digital assets in a decentralized environment.